Threads

Process characteristics

A process may execte other processes through cloning UNIX, `fork` or replacing the current image with another image UNIX, `exec`
Each process has its own address space and a single execution thread (a single program counter)

Cloning involves:

- a significant increare in the memory used
- creation time overhead
- synchronization and data transger
  - no cost or minimal for almost indipendent processes
  - high cost for cooperating processes
- management of multiple processes requires
  - complex scheduling
  - expensive context switching operations

Threads

There are several cases where it would be useful to have threads

- Lower creation and management costs
- A single address space. Thread 1 and thread 2, can access the same varaibles.
- Multiple execution threads (concurrency) within that address space

They where standarized in 1996:

- The thread model allows a program to control multiple different flows of operations (scheduled and executed independently) that overlap in time
- Each flow of operations is referred to as a thread
- Creation and control over these flows is achieved by making calls to the POSIX Threads API.
- A thread can share its address space with other threads

(Process = unit that groups resources)
(Thread = CPU scheduling unit)
(Thread=lightweight process)

Data

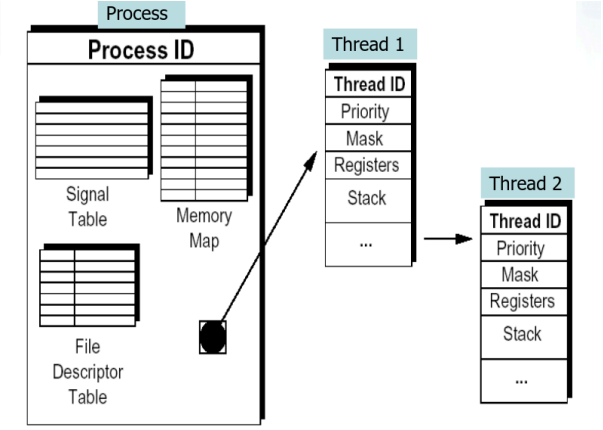
Shared data: (with other threads of the same process)

- code section (also in processes)
- data section (global variables, file descriptors, shared data etc) (only in threads)
- operating system resources (e.g. signals) (also processes)

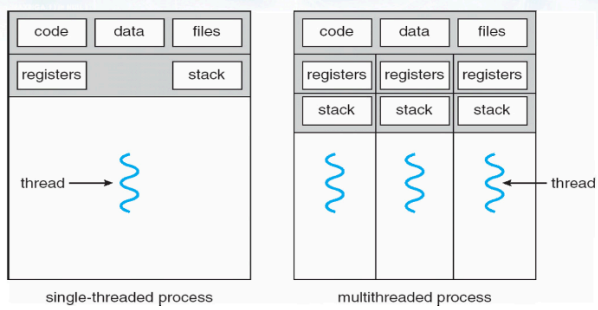
Private data:

- Program counter and hardware registers

- Stack (i.e. local variables and execution history)



Signal table: maps each signal to a given memory address
Memory map: says which are of memory can be used by the process and which not
File descripto table: list of open file descriptor
Multiple thread (arrow)



Advantages

The use of threads allows

- Shorter response time
  - Creating a thread it is 10-100 times faster than creating a process
  - Example
    - The creation of 50000 jobs (fork) takes 10 seconds (real time)
    - The creation of 50000 thread (pthread_create) takes 0.8 seconds (real time)
- Shared resources
  - Processes can share data only with special techniques
  - Threads share data automatically
- Lower costs for resource management
  - Allocate memory to a process is expensive
  - Threads use the same section of code and/or data to serve more clients
- Increased scalability
  - The advantages of multi-threaded programming increase in multi-processor systems
  - In multi-core systems (different calculation units per processor) threads allow easily implementing concurrent programming paradigms based on
    - Task separation (pipelining)
    - Data partitioning (same task on data blocks)

Disadvantages

- There is no protection for threads
  - They are executed in the same address space and OS protection is impossible or unnecessary
  - If the threads are not synchronized, access to shared data is not thread safe
- There is not a parent-child hierarchical relationship between threads
  - To the creating thread is normally returned the identifier of the created thread, but this does not imply a hierarchical relationship

- All threads are "equal"

Concurrency with theards

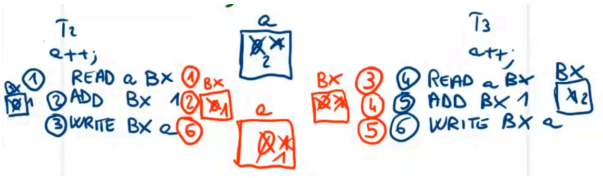
Optimize the following code segment that performs the scalar product of four huge dimension vectors (v1, v2, v3, v4)

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000
#define NUM_THREADS 4

// Global Vectors
float v1[N], v2[N], v3[N], v4[N];
float global_sum = 0;
pthread_mutex_t sum_mutex; // Protects the global_sum

typedef struct {
    int start;
    int end;
} thread_data_t;

void* scalar_product_part(void* arg) {
    thread_data_t *data = (thread_data_t *)arg;
    float local_sum = 0;

    // 1. Calculate local partial sum (No mutex needed here)
    for (int i = data->start; i < data->end; i++) {
        local_sum += (v1[i] * v2[i]) + (v3[i] * v4[i]);
    }

    // 2. Safely add to global sum using a Mutex
    pthread_mutex_lock(&sum_mutex);
    global_sum += local_sum;
    pthread_mutex_unlock(&sum_mutex);

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    thread_data_t thread_args[NUM_THREADS];

    pthread_mutex_init(&sum_mutex, NULL);

    // Initialize vectors with dummy data (omitted for brevity)

    // Create threads
    int chunk_size = N / NUM_THREADS;
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_args[i].start = i * chunk_size;
        thread_args[i].end = (i == NUM_THREADS - 1) ? N : (i + 1) * chunk_size;
        pthread_create(&threads[i], NULL, scalar_product_part, &thread_args[i]);
    }

    // Join threads
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Final Scalar Product: %f\n", global_sum);
    pthread_mutex_destroy(&sum_mutex);
    return 0;
}
```

Example:

```
int a
th_sum(int *a){
     *a++;
}

main(){ //main thread
     a=0
     phthread_create(...,th_sum,...); //T2
     phthread_create(...,th_sum,...); //T3
     sleep(10); //makes T1 wait on T2 and T3
     printf("%d", a);
}
```

What is the output?
0; if T2 and T3 take longer than 10 seconds
1; if T2 take a bit of time to write to a, making T3 read a=0 instead of a=1, so when T3 writes, it writes a=1
2; if T2 finishes before T3 and T3 before T1's sleep



The solution is to not act on critical regions concurently, creating sequential access.

Multithread programming models

Three multithread programming models exist:

- Kernel-level thread
  - Thread implemented at kernel-level
  - The kernel directly supports the thread concept
- User-level thread
  - Thread implemented at user-level
  - The kernel is not aware that threads exist
- Mixed or hybrid solution
  - The operating system provides both user-level and kernel threads

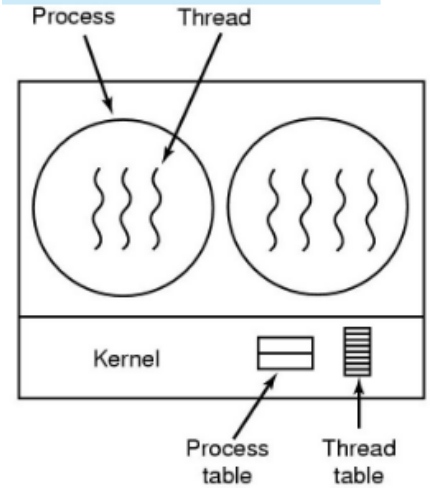
Kernel-level threads

- Threads are managed by the kernel
- The OS
  - Manipulates both processes and threads
  - Is aware of the existence of threads
  - Provides adequate support for their handling
    - All the operations on threads (creation, synchronization, etc.) are performed through system calls

The operating system, for each thread, keeps information similar to those it maintains for each process

- Thread table
- Thread Control Block (TCB) for each active thread

Managed information is global within the whole OS



Advantages

- Since the operating system is aware of threads:
  - It can select thread to schedule among the ready threads of all processes. Since it has global view of all threads of all processes
  - It has the possibly of allocating more CPU time to processes with many threads than to processes with few threads
- Effectiveness in applications that perform often blocking calls (e.g., blocking read)
  - Ready threads can be scheduled even if they belong to the same task of a thread that called a blocking system call
  - That is, if one thread blocks, it is always possible to execute another thread in the same process (or in another) because the OS checks all the threads of all the processes
- It allows an effective parallelism
  - Multiple threads can be executed in a multiprocessor system

Disadvantages

- Due to the transition to kernel mode the management is relatively slow and inefficient
  - Expensive context switching
  - Handling times hundreds of times slower than necessary
- Limitation on the maximum number of threads
  - The OS must control the number of generated threads
- Expensive information management (thread table and TCB)

User-level threads

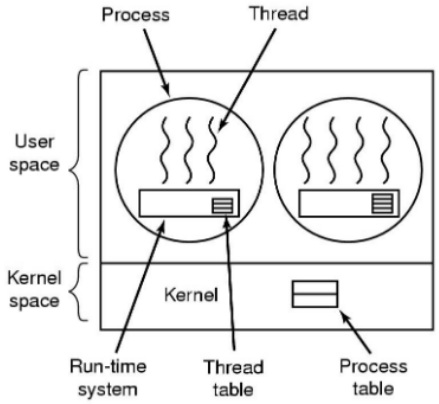
The thread package is fully implemented in the user space as a seto of functions:

- The kernel is not aware about threads, it manages only processes
- Threads are managed run-time through a library
  - Support by means of a set of functions, called from user-space
  - Thread creation, synchronization, scheduling, etc. do not require kernel intervention
  - Function are used, not system calls

Each process needs a personal table of running threads:

- Needed information is less than the management at the kernel-level
  - Smaller TCBs

- Local visibility of information (within the process)



Advantages

- Can be implemented on top of any kernel, even in systems that do not support threads natively
- Do not require modifications to the OS
- Efficient management
  - Fast context switching between threads of the same task
  - Efficient data manipulation
  - Hundreds of times faster than kernel threads
- Allow the programmer to generate the desired number of threads
  - It might be possible to think of scheduling/custom management of threads within each process

Disadvantages

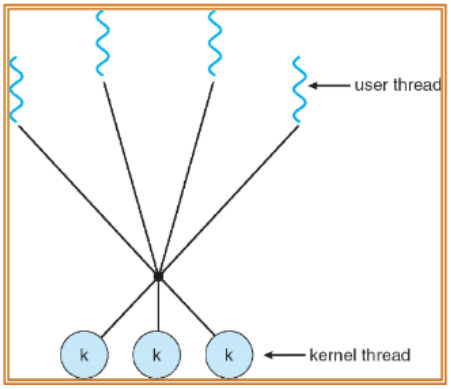
- The operating system does not know the existence of threads
- Inappropriate or inefficient choices can be made
  - The OS could schedule a process whose running thread could do a blocking operation
  - In this case, the whole process could be blocked even if inside it several other threads could be executed
- Information should be communicated between kernel and run-time user-level manager
- Without this communication mechanism
  - Exist only on running thread for each task even in a multiprocessor system
  - There is no scheduling within a single process, i.e., interrupts do not exist within a single process
  - If a running thread does not release the CPU, it cannot be blocked
- The scheduler must map user threads to the single kernel thread
  - If the kernel thread blocks, all the user-level threads are blocked
  - There is no true thread-level parallelism without handling multiple threads at the kernel-level

Hybrid thread

This is used by all OSs
One of the multi-thread programming problems is to define the relationship between user-level threads and kernel-level threads.

- The basic idea is to have m user threads and to map them to n kernel threads

- Typically, n < m



The hybrid implementation attempts to combine the advantages of both approaches

- The user decides the number of its user-level threads, and the number of kernel-threads on which they must be mapped
- The kernel is aware only of the kernel thread and only manages those threads
- Each kernel thread can be used in turn by several user threads

Coexistence between processes and threads

The coexistence of processes and threads causes various kinds of problems related to

- Signals management
  - With T defines the behavior at the reception of a signal, and which T receives signals?
- Cloning (fork) a process
  - A fork performs the duplication of all the thread (forkall), or it only duplicates the thread that invokes it (fork1)?
- The substitution of a program with another (exec)
  - Does an exec only replace the thread that executes the exec, or does it duplicate all threads in the process?

Signals management

- Definition of the behaviour at the reception of a signal
  - Each thread can define the behavior of the process at the reception of a signal
  - Different threads can define conflicting actions
- Each signal is delivered to a single thread within the process
  - If a signal is specifically associated with a thread, it is delivered to that thread (`pthread_sigmask`)
  - Generic signals are delivered to an arbitrary thread

Use of the system call `fork`

- In a multi-thread process, a fork duplicates only the thread that calls the fork
- The private data owned by the threads not duplicated by the fork may not be "manageable" by the single duplicated thread
  - To manage correctly the state of the process after the fork, UNIX provides specific system calls, e.g., `pthread_atfork`

Use of the system call `exec`

- An `exec` performed by a thread substitutes with the new program the whole process (not only the thread that executes the exec)
- If a duplicated thread executes and exec immediately after the execution of a fork, the states of the other threads before the fork are not important, because they are not duplicated by the fork

Pthread library

It provides the programmer the intergace to use the threads.
POSIX threads or Pthreads is the standard UNIX library for threads. Defined for C UNIX, but availavle for other platform and languages.
Its implementation depends on the platform.

A thread is a function that is executed in concurrency with the main thread.
A process with multiple threads= a set of indipendently executing functions that share the process resources.

The pthread library allows:

- Creating and manipulating threads
- Synchronizing threads
- Protection of resources shared by threads
- Thread scheduling
- Destroying thread

It defines more than 60 functions. All these functions have a `pthread_*` prefix:

```
pthread_equal, pthread_self,
pthread_create, pthread_exit,
pthread_join, pthread_cancel,
pthread_detach, ...
```

Library linkage

The Phtread system calls are defined in `pthreads.h`.
It is important to link the `pthread` library while compiling .c programs:

```
gcc -Wall -g -o <exeName> <file.c> -lpthread
```

Thread identifier

A thread is uniquely identified by a type identifier `pthread_t`. Similar to the PID of a process (`pid_t`).
The type `pthread_t` is opaque. Its definition is implementation dependent. It can be used only by functions specifically defined in Pthreads, Its not possible to compare directly two identifiers or print their values.
It has meaning only within the process where the thread is executed, the PID is the systne wude identifier (of the process, not thread).

pthread_equal()

Compares two thread identifiers.

```
int pthread_equal (
     pthread_t tid1,
     pthread_t tid2
);
```

`tid1` & `tid2`: thread identefiers
return:

- non zero if equal
- 0 if not equal

pthread_self()

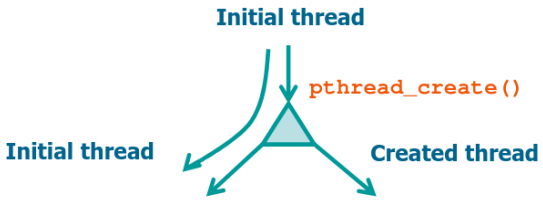
```
pthread_t pthread_self (void);
```

`return`: thread identifier of the calling thread

pthread_create()

At the beginning, a program consists of one process and one thread.
`pthread_create` allows creating a new thread. The maximum number of thread that can be created is undefined and

implementation dependent




```
int pthread_create (pthread_t *tid, const pthread_attr_t *attr, void *(*startRoutine)(void *), void
*arg);
```

`tid` : identifier of the generated thread
`attr` : thread attributes (`NULL` by default)
`startRoutine` : C function executed by the thread
`arg` : Argument passed to the start routine
`return` :

- 0 on success
- Error code on failure

pthread_exit()

A whole process with al its threads terminates if:

- Its thread calls `exit` (or `_exit` or `_Exit`)
- The main thread execute return
- The main thread receives a signal whose action is to terminate

A single thread can termiante (without affecting the other process threads):

- Executing return from its start function
- Executing pthread_exit
- Receiving a cancellation request performed by another thread using pthread_cancel

pthread_exit() allows a thread to terminate returning a termination status

```
void pthread_exit (
     void *valuePtr
);
```

`valuePtr` : value is kept by the kernal until a thread calls `pthread_join()` , then the value become available to the thread that calls pthread_join().

buggy code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

#define NUM_THREADS 8

void *PrintHello ( void *threadid) {
     int *id_ptr, taskid;
     //sleep(1);
     id_ptr = (int *) threadid;
     taskid = *id_ptr;
     fprintf (stdout, "Hello from thread %d\n", taskid);
     pthread_exit (NULL);
}

int main (int argc, char *argv[]){
```

```
      pthread_t threads[NUM_THREADS];
      int rc, t;
      for (t=0; t<NUM_THREADS; t++) {
           printf("Creating thread %d\n", t);
           rc = pthread_create (&threads[t], NULL, PrintHello, (void *) &t);
           if (rc) {
                printf("ERROR; return code from pthread_create() is %d\n", rc);
                exit(-1);
           }
       }
      pthread_exit(NULL);
 }
```

The scheduler doesnt know your code

```
polycore% gcc e06-ArgBuggy.c
polycore% ./a.out
Creating thread 0
Creating thread 1
Creating thread 2
Creating thread 3
Hello from thread 3
Hello from thread 4
Hello from thread 3
Creating thread 4
Hello from thread 4
Creating thread 5
Hello from thread 5
Hello from thread 6
Creating thread 6
Creating thread 7
Hello from thread 8
Hello from thread 8
```

pthread_join()

At a threads creation, it self can be declared

- joinable
  - Another thread may "wait" (pthread_join) for its termination, and collect its exit status
  - its termination status is retained until another thread performs a `pthread_joun` for that thread
- detached
  - No thread can explicitly wait for its termination
  - its termination status is immeidately released

In any case a thread calling `pthread_join` waits until the required thread calls `phread_exit` .

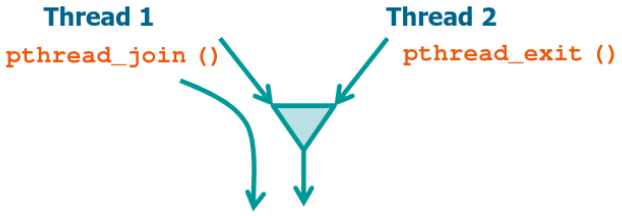
Used by a thread to wait the termination of another thread

```
 int pthread_join (
      pthread_t tid,
      void **valuePtr
 );
```

`tid` : identifeira of the waited-for thread
`ValuePtrL` : will obtain the value returned by thread `tid`
`return` :

- 0 on success
- Error code on failure
  - if the thread was detached (can happen that it terminates corectly non the less)

- returns `EINVAL` or `ESRCH`




```
int myglobal;
void *threadF (void *arg) {
     int *argc = (int *) arg;
     int i, j;
     for (i=0; i<20; i++) {
          j = myglobal;
          j = j + 1;
          printf ("t");
          if (*argc > 1) sleep (1);
          myglobal = j;
     }
     printf ("(T:myglobal=%d)", myglobal);
     return NULL;
}

int main (int argc, char *argv[]) {
     pthread_t mythread;
     int i;
     pthread_create (&mythread, NULL, threadF, &argc);
     for (i=0; i<20; i++) {
          myglobal = myglobal + 1;
          printf ("m");
          sleep (1);
     }
     pthread_join (mythread, NULL);
     printf ("(M:myglobal=%d)", myglobal);
     exit (0);
}
```

pthread_cancel

Terinates the target thread (The effect is similar to a call to `pthread_exit(PTHREAD_CANCELED)` performed by the target thread)
The thread calling pthread_cancel does not wait for termination of the target thread (it continues immediately after the calling)

```
int pthread_cancel (pthread_t tid);
```

`tid` : target thread identifier
`return` :

- 0 on success
- Error code on failure

pthread_detach()

Declares thread `tid` as detached

```
int pthread_detach (pthread_t tid);
```

`tid` : thread identifier
`return` :

- 0 on success
- Error code on failure

Exercise:

Given a text file, with an undefined number of characters, passed as an argument of the command line

- Implement a concurrent program using three threads (T1, T2, T3) that process the file content in pipeline
- T1: Read from file the next character
- T2: Transforms the character read by T1 in uppercase
- T3: Displays the character produced by T2 on standard output

lets say its "ciao"

time	R	T	W
1	c		
2	i	C	
3	a	I	C
4	o	A	I
5		O	A
6			O

So the operation would take 6 seconds, instead of:

time	R	T	W
1	c		
2	i		
3	a		
4	o		
5		C	
6		I	
7		A	
8		O	
9			C
10			I
11			A
12			O